JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class LineNumberReader

java.lang.Object
    java.io.Reader
        java.io.BufferedReader
            java.io.LineNumberReader

All Implemented Interfaces:

Closeable, AutoCloseable, Readable

```
public class LineNumberReader
extends BufferedReader
```

A buffered character-input stream that keeps track of line numbers. This class defines methods `setLineNumber(int)` and `getLineNumber()` for setting and getting the current line number respectively.

By default, line numbering begins at 0. This number increments at every line terminator as the data is read, and at the end of the stream if the last character in the stream is not a line terminator. This number can be changed with a call to `setLineNumber(int)`. Note however, that `setLineNumber(int)` does not actually change the current position in the stream; it only changes the value that will be returned by `getLineNumber()`.

A line is considered to be terminated by any one of a line feed ('\n'), a carriage return ('\r'), or a carriage return followed immediately by a linefeed, or any of the previous terminators followed by end of stream, or end of stream not preceded by another terminator.

Since:

1.1

Field Summary

Fields declared in class Reader
lock

Constructor Summary

Constructors

Constructor	Description
`LineNumberReader(Reader in)`	Create a new line-numbering reader, using the default input-buffer size.

LineNumberReader(Reader in, int sz)	Create a new line-numbering reader, reading characters into a buffer of the given size.

Method Summary

All Methods | Instance Methods

Modifier and Type	Method	Description
int	**getLineNumber**()	Get the current line number.
void	**mark** (int readAheadLimit)	Mark the present position in the stream.
int	**read**()	Read a single character.
int	**read**(char[] cbuf, int off, int len)	Reads characters into a portion of an array.
String	**readLine**()	Read a line of text.
void	**reset**()	Reset the stream to the most recent mark.
void	**setLineNumber** (int lineNumber)	Set the current line number.

Methods declared in class BufferedReader

close, lines, markSupported, ready

Methods declared in class Reader

nullReader, of, read, read, readAllAsString, readAllLines, skip, transferTo

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Constructor Details

LineNumberReader

```
public LineNumberReader(Reader in)
```

Create a new line-numbering reader, using the default input-buffer size.

Parameters:

`in` - A Reader object to provide the underlying stream

LineNumberReader

```
public LineNumberReader(Reader in,
                        int sz)
```

Create a new line-numbering reader, reading characters into a buffer of the given size.

Parameters:

`in` - A Reader object to provide the underlying stream

`sz` - An int specifying the size of the buffer

Method Details

setLineNumber

```
public void setLineNumber(int lineNumber)
```

Set the current line number.

Parameters:

`lineNumber` - An int specifying the line number

See Also:

`getLineNumber()`

getLineNumber

```
public int getLineNumber()
```

Get the current line number.

Returns:

The current line number

See Also:

`setLineNumber(int)`

read

```
public int read()
         throws IOException
```

Read a single character. Line terminators are compressed into single newline ('\n') characters. The current line number is incremented whenever a line terminator is read, or when the end of the stream is reached and the last character in the stream is not a line terminator.

Overrides:

`read` in class `BufferedReader`

Returns:

The character read, or -1 if the end of the stream has been reached

Throws:

`IOException` - If an I/O error occurs

read

```
public int read(char[] cbuf,
                int off,
                int len)
         throws IOException
```

Reads characters into a portion of an array. This method will block until some input is available, an I/O error occurs, or the end of the stream is reached.

If `len` is zero, then no characters are read and `0` is returned; otherwise, there is an attempt to read at least one character. If no character is available because the stream is at its end, the value `-1` is returned; otherwise, at least one character is read and stored into `cbuf`.

Line terminators are compressed into single newline ('\n') characters. The current line number is incremented whenever a line terminator is read, or when the end of the stream is reached and the last character in the stream is not a line terminator.

Overrides:

`read` in class `BufferedReader`

Parameters:

`cbuf` - Destination buffer

`off` - Offset at which to start storing characters

`len` - Maximum number of characters to read

Returns:

The number of characters read, or -1 if the end of the stream has been reached

Throws:

`IndexOutOfBoundsException` - If `off` is negative, or `len` is negative, or `len` is greater than `cbuf.length - off`

`IOException` - If an I/O error occurs

readLine

```
public String readLine()
                throws IOException
```

Read a line of text. Line terminators are compressed into single newline ('\n') characters. The current line number is incremented whenever a line terminator is read, or when the end of the stream is reached and the last character in the stream is not a line terminator.

Overrides:

`readLine` in class `BufferedReader`

Returns:

A String containing the contents of the line, not including any line termination characters, or `null` if the end of the stream has been reached

Throws:

`IOException` - If an I/O error occurs

See Also:

`Files.readAllLines(Path, Charset)`

mark

```
public void mark(int readAheadLimit)
          throws IOException
```

Mark the present position in the stream. Subsequent calls to reset() will attempt to reposition the stream to this point, and will also reset the line number appropriately.

Overrides:

`mark` in class `BufferedReader`

Parameters:

`readAheadLimit` - Limit on the number of characters that may be read while still preserving the mark. After reading this many characters, attempting to reset the stream may fail.

Throws:

`IOException` - If an I/O error occurs

reset

```
public void reset()
           throws IOException
```

Reset the stream to the most recent mark.

Overrides:

`reset` in class `BufferedReader`

Throws:

`IOException` - If the stream has not been marked, or if the mark has been invalidated

---

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.